AWS Secrets Manager: Purpose and Role in Secure Credential Management

Week 4, Task 14 | Phase A: Theory & Conceptual Overview | Titilayo Yusuf | 27 October 2026

1. Purpose

AWS Secrets Manager is a managed service for storing, rotating, retrieving and auditing secrets such as database credentials, API keys and OAuth tokens. Applications request a secret at runtime through an authenticated API call instead of carrying it in code or configuration. This moves credentials out of source code and gives security teams one place to control who can read a secret, how often it changes, and when it was accessed.

2. How Secrets Manager encrypts data with KMS

- **Envelope encryption:** every secret value is encrypted with a unique data key. That data key is itself encrypted under an AWS KMS key and stored with the secret's metadata. Plaintext keys are never stored.
- **Key choice:** the default AWS managed key (`aws/secretsmanager`) or a customer managed key (CMK). A CMK adds control over the key policy, rotation and audit, and is required to share a secret across accounts.
- **Two permission layers:** reading a secret needs `secretsmanager:GetSecretValue` and the right to use the KMS key (`kms:Decrypt` via key policy or grant). Either one missing means access is denied.
- **Encryption at rest vs in transit: At rest**, values are stored encrypted with AES-256 through KMS. **In transit**, all API calls use TLS and are signed with SigV4; a VPC interface endpoint (PrivateLink) can keep traffic off the public internet. Decryption happens only inside the service after authorization, and returned values are protected by TLS.

3. Environment variables vs managed secrets

Aspect	Environment variables	Secrets Manager
Storage	Plaintext in config, task definition or process environment	Encrypted with KMS; fetched at runtime
Exposure	Visible in console, logs, crash dumps, `docker inspect`, child processes	Returned only to authorized IAM principals over TLS
Rotation	Manual; needs redeploy	Scheduled and automatic via Lambda; versioned
Access control & audit	Anyone who can read the config sees the value; no per-read record	IAM and resource policies per secret; every call logged in CloudTrail

4. Secret lifecycle: creation, rotation, retrieval, deletion

- **Creation:** `CreateSecret` stores the value, selects the KMS key, and can attach tags and a resource policy. The first version gets the staging label `AWSCURRENT`.
- **Rotation:** a rotation Lambda runs on a schedule through four steps (`createSecret, setSecret, testSecret, finishSecret`). The new version is promoted to `AWSCURRENT` and the old one becomes `AWSPREVIOUS`, so apps keep working.
- **Retrieval:** workloads call `GetSecretValue` using an IAM role (never long-lived keys), ideally through a caching client so the secret is not fetched on every request.
- **Deletion:** `DeleteSecret` schedules removal after a recovery window of 7 to 30 days (default 30), during which `RestoreSecret` works. Force delete skips the window and is irreversible.

5. Restricting access to specific ARNs with IAM

Least privilege means granting only the needed action on only the needed secret. Secret ARNs end in a random six-character suffix, so policies use an exact ARN or a trailing wildcard.

```
{ "Effect": "Allow",
  "Action": "secretsmanager:GetSecretValue",
  "Resource": "arn:aws:secretsmanager:us-east-1:111122223333:secret:prod/app/db-*" }
```

Pair this with `kms:Decrypt` on the specific CMK. Avoid `Resource: "*"`. Resource policies and condition keys (for example `aws:PrincipalTag` or `aws:SourceVpce`) tighten access further.

6. Risks of hardcoding credentials

- **Source-control leakage:** secrets committed to Git persist in history and are found by tools like TruffleHog and GitLeaks, as seen in Task 13.
- **Long-lived, unrotated credentials:** a leaked key stays valid for months; changing it requires a code change and redeploy.
- **Broad blast radius:** the same credential is often copied across environments and developers, so one leak exposes production.
- **Spread into logs, images and build artifacts:** values leak through CI logs, container layers and backups outside normal access controls.
- **No attribution or audit trail:** a hardcoded key cannot show who used it or when, which weakens detection and compliance evidence.

Takeaway: Secrets Manager replaces static, widely copied credentials with encrypted, access-controlled, rotating and audited secrets retrieved at runtime.